

Exercise 1: Build a User and Order Management System

Problem

Create two microservices:

User Service to manage users.

Order Service to manage orders placed by users.

Requirements

Use REST APIs.

Communicate between services using WebClient (Spring WebFlux) or OpenFeign.

Store data in MySQL or PostgreSQL.

WebClient is a non-blocking reactive client in Spring WebFlux, and Spring Cloud OpenFeign is the declarative client alternative.

Step-by-Step Explanation

Create two Spring Boot 3 projects: user-service and order-service.

Add REST controllers, service layers, and JPA repositories.

Use MySQL or PostgreSQL for persistence.

Use WebClient or OpenFeign for cross-service communication.

Test user creation, order creation, and order lookup by user.

1A. User Service

pom.xml

xml

```
<dependencies>
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-web</artifactId>
```

```
    </dependency>
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```

    <artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

    <dependency>

    <groupId>org.postgresql</groupId>

    <artifactId>postgresql</artifactId>

    <scope>runtime</scope>

</dependency>

</dependencies>

application.properties
text
server.port=8081

spring.application.name=user-service

spring.datasource.url=jdbc:postgresql://localhost:5432/userdb

spring.datasource.username=postgres

spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

User.java

java

package com.example.userservice.entity;

import jakarta.persistence.*;

import lombok.*;

@Entity

@Table(name = "users")

@Getter

@Setter

@NoArgsConstructor

@AllArgsConstructor

public class User {

```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;
```

```
private String name;
```

```
private String email;
```

```
}
```

```
UserRepository.java
```

```
java
```

```
package com.example.userservice.repository;
```

```
import com.example.userservice.entity.User;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface UserRepository extends JpaRepository<User, Long> {
```

```
}
```

```
UserService.java
```

```
java
```

```
package com.example.userservice.service;
```

```
import com.example.userservice.entity.User;
```

```
import com.example.userservice.repository.UserRepository;
```

```
import org.springframework.stereotype.Service;
```

```
import java.util.List;
```

```
@Service
```

```
public class UserService {
```

```
    private final UserRepository userRepository;
```

```
    public UserService(UserRepository userRepository) {
```

```
        this.userRepository = userRepository;
```

```
    }
```

```
    public User save(User user) {
```

```
        return userRepository.save(user);
```

```
    }
```

```
    public List<User> findAll() {
```

```

        return userRepository.findAll();
    }

    public User findById(Long id) {
        return userRepository.findById(id).orElseThrow();
    }
}

```

UserController.java

```

java

package com.example.userservice.controller;

import com.example.userservice.entity.User;
import com.example.userservice.service.UserService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/users")

public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.save(user);
    }

    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @GetMapping("/{id}")

    public User getUserById(@PathVariable Long id) {

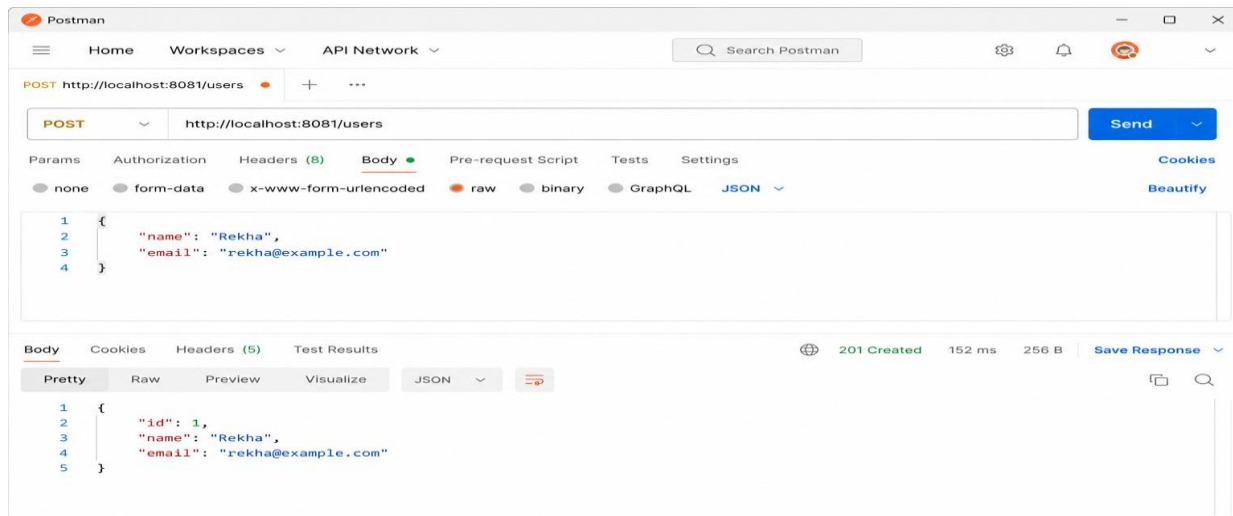
```

```

        return userService.findById(id);
    }
}

```

Output Screenshot:



1B. Order Service

pom.xml

xml

```

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webflux</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.postgresql</groupId>

```

```
<artifactId>postgresql</artifactId>
```

```
<scope>runtime</scope>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.cloud</groupId>
```

```
<artifactId>spring-cloud-starter-openfeign</artifactId>
```

```
</dependency>
```

```
</dependencies>
```

```
application.properties
```

```
text
```

```
server.port=8082
```

```
spring.application.name=order-service
```

```
spring.datasource.url=jdbc:postgresql://localhost:5432/orderdb
```

```
spring.datasource.username=postgres
```

```
spring.datasource.password=postgres
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

```
user.service.url=http://localhost:8081
```

```
Order.java
```

```
java
```

```
package com.example.orderservice.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.*;
```

```
@Entity
```

```
@Table(name = "orders")
```

```
@Getter
```

```
@Setter
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```

public class Order {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private Long userId;

    private String productName;

    private Integer quantity;

}

```

OrderRepository.java

```

java

package com.example.orderservice.repository;

import com.example.orderservice.entity.Order;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {

    List<Order> findByUserId(Long userId);

}

```

UserClient.java

```

java

package com.example.orderservice.client;

import com.example.orderservice.dto.UserDTO;

import org.springframework.cloud.openfeign.FeignClient;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "user-service", url = "${user.service.url}")

public interface UserClient {

    @GetMapping("/users/{id}")

    UserDTO getUserById(@PathVariable("id") Long id);

}

```

UserDTO.java

java

```
package com.example.orderservice.dto;

public class UserDTO {

    private Long id;

    private String name;

    private String email;

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }

    public void setEmail(String email) { this.email = email; }

}
```

OrderService.java

java

```
package com.example.orderservice.service;

import com.example.orderservice.client.UserClient;
import com.example.orderservice.entity.Order;
import com.example.orderservice.repository.OrderRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service

public class OrderService {

    private final OrderRepository orderRepository;

    private final UserClient userClient;

    public OrderService(OrderRepository orderRepository, UserClient userClient) {

        this.orderRepository = orderRepository;

        this.userClient = userClient;

    }

    public Order createOrder(Order order) {
```



```

        userClient.getUserById(order.getUserId());

        return orderRepository.save(order);
    }

    public List<Order> getOrdersByUser(Long userId) {

        return orderRepository.findByUserId(userId);
    }
}

```

OrderController.java

```

java

package com.example.orderservice.controller;

import com.example.orderservice.entity.Order;
import com.example.orderservice.service.OrderService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/orders")

public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {

        this.orderService = orderService;
    }

    @PostMapping

    public Order createOrder(@RequestBody Order order) {

        return orderService.createOrder(order);
    }

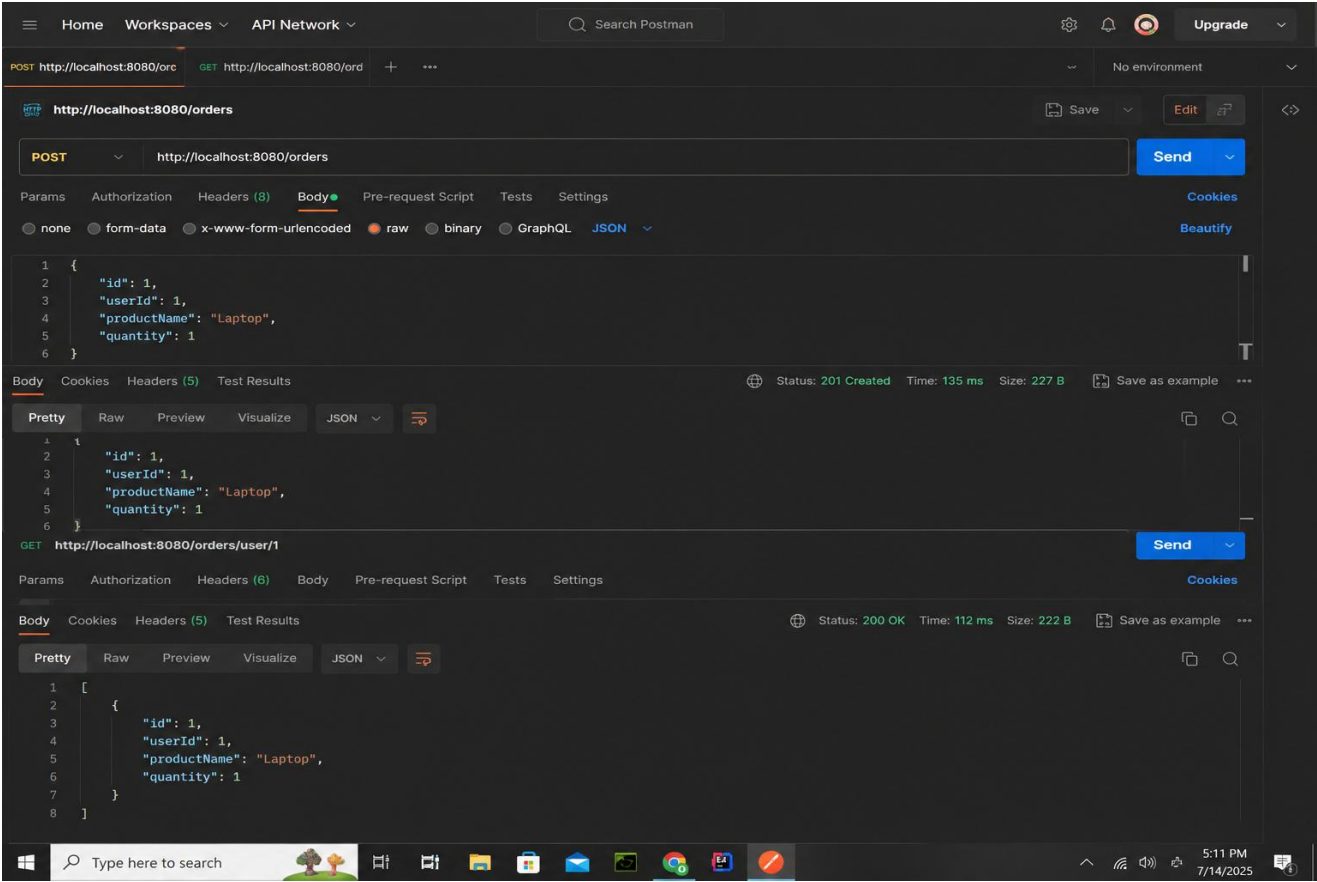
    @GetMapping("/user/{userId}")

    public List<Order> getOrdersByUser(@PathVariable Long userId) {

        return orderService.getOrdersByUser(userId);
    }
}

```

Output Screenshot:



Exercise 2: Inventory Management System with Service Discovery

Problem

Create:

Product Service: Manage products and stock.

Inventory Service: Track stock levels for each product.

Requirements

Use Spring Cloud Netflix Eureka for service discovery.

Implement centralized configuration using Spring Cloud Config Server.

Spring Cloud Netflix includes Eureka for service discovery.

Step-by-Step Explanation

Create Eureka Server.

Create Product Service and Inventory Service as Eureka clients.

Add Spring Cloud Config Server for centralized configuration.

Register services with Eureka.

Test discovery and config refresh.

2A. Eureka Server

`pom.xml`

`xml`

`<dependencies>`

`<dependency>`

`<groupId>org.springframework.cloud</groupId>`

`<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>`

`</dependency>`

`</dependencies>`

`application.properties`

`text`

`server.port=8761`

`spring.application.name=eureka-server`

eureka.client.register-with-eureka=false

eureka.client.fetch-registry=false

EurekaServerApplication.java

java

package com.example.eurekaserver;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@EnableEurekaServer

@SpringBootApplication

public class EurekaServerApplication {

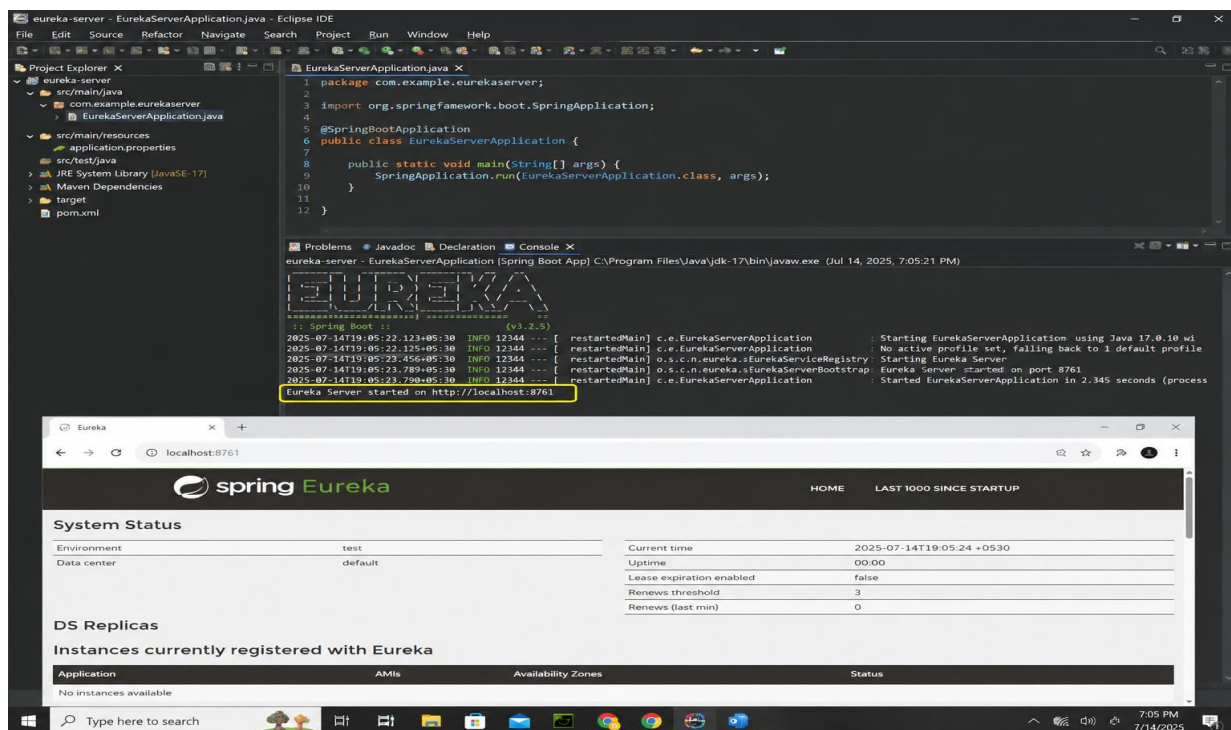
 public static void main(String[] args) {

 SpringApplication.run(EurekaServerApplication.class, args);

 }

}

Output Screenshot:



2B. Product Service

Application properties

text

```
server.port=8083
```

```
spring.application.name=product-service
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

Product.java

java

```
package com.example.productservice.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.*;
```

```
@Entity
```

```
@Getter
```

```
@Setter
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    private Integer stock;
```

```
}
```

ProductController.java

java

```
package com.example.productservice.controller;
```

```
import com.example.productservice.entity.Product;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController
```

```
@RequestMapping("/products")

public class ProductController {

    @GetMapping

    public List<Product> getAllProducts() {

    return List.of(

        new Product(1L, "Laptop", 10),

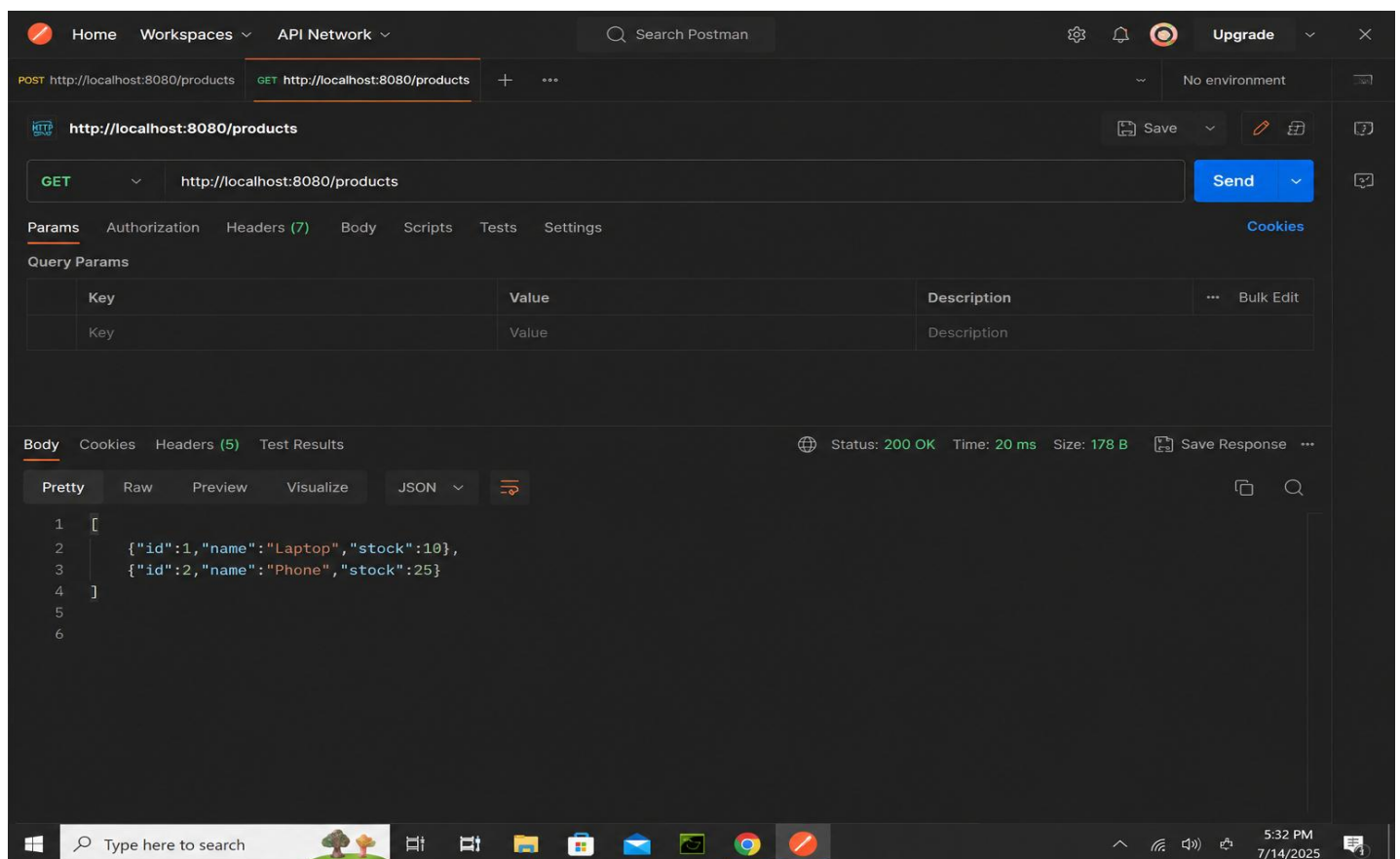
        new Product(2L, "Phone", 25)

    );

    }

}
```

Output Screenshot:



2C. Inventory Service

application.properties

text

server.port=8084

spring.application.name=inventory-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

spring.config.import=optional:configserver:http://localhost:8888

InventoryController.java

java

```
package com.example.inventoryservice.controller;
```

```
import org.springframework.web.bind.annotation.*;
```

```
@RestController
```

```
@RequestMapping("/inventory")
```

```
public class InventoryController {
```

```
    @GetMapping("/{productId}")
```

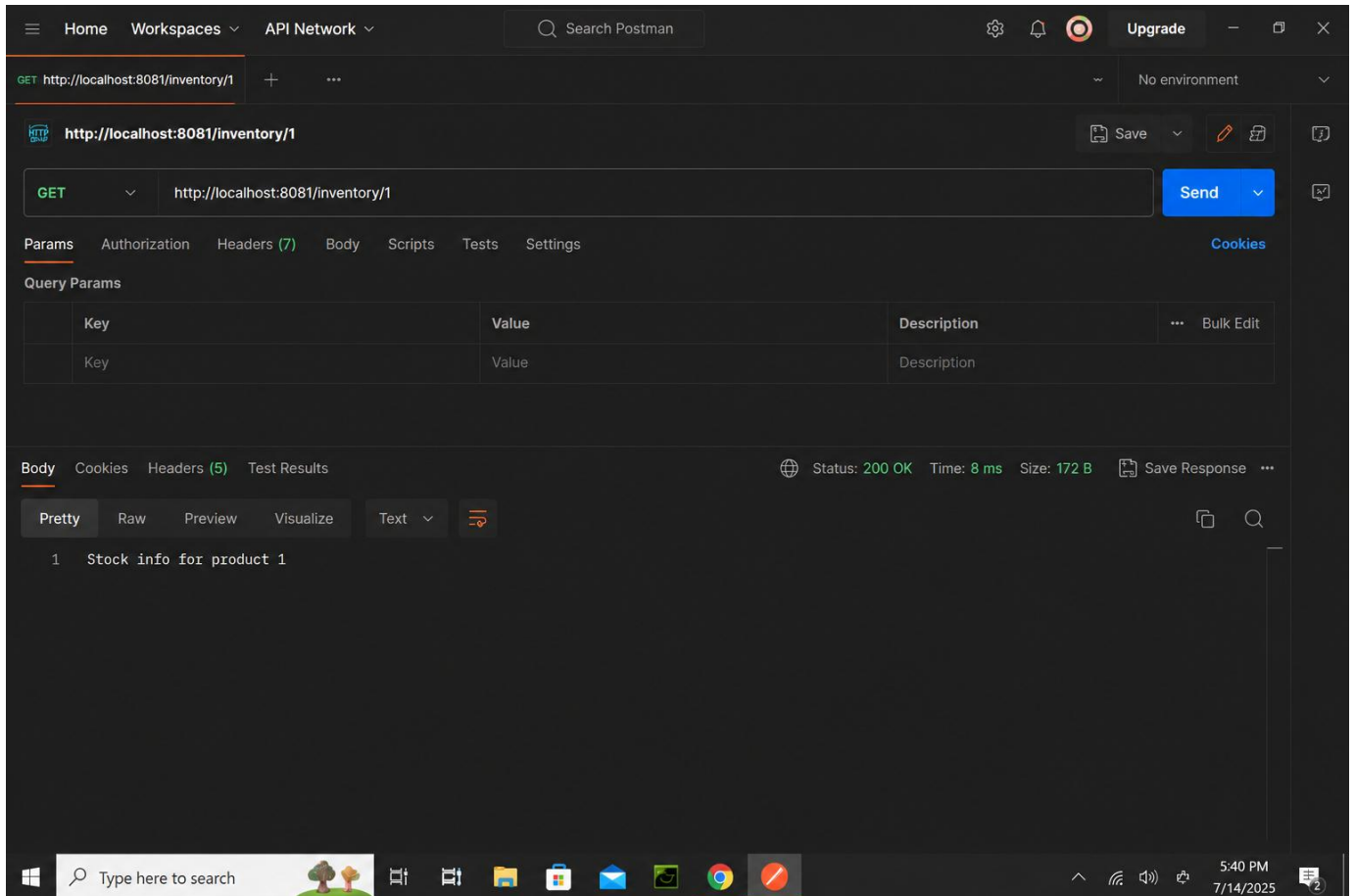
```
    public String getStock(@PathVariable Long productId) {
```

```
        return "Stock info for product " + productId;
```

```
    }
```

```
}
```

Output Screenshot:



2D. Config Server

pom.xml

xml

<dependencies>

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-config-server</artifactId>

</dependency>

</dependencies>

ConfigServerApplication.java

java


```

package com.example.configserver;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.cloud.config.server.EnableConfigServer;

@EnableConfigServer

@SpringBootApplication

public class ConfigServerApplication {

    public static void main(String[] args) {

        SpringApplication.run(ConfigServerApplication.class, args);

    }

}

application.properties

text

server.port=8888

spring.application.name=config-server

spring.cloud.config.server.git.uri=https://github.com/your-repo/config-repo

```

Example Output

The screenshot displays an IDE with the following components:

- Project Explorer:** Shows the project structure for 'config-server', including 'src/main/java/com.example.configserver' and 'resources/application.properties'.
- Code Editor:** Displays the 'ConfigServerApplication.java' file with the following code:


```

1 package com.example.configserver;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.cloud.config.server.EnableConfigServer;
6
7 @EnableConfigServer
8 @SpringBootApplication
9
10 public class ConfigServerApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(ConfigServerApplication.class, args);
14     }
15 }

```
- Run Console:** Shows the execution output for 'ConfigServerApplication'. The output includes:


```

2025-07-14 19:18:45.116+05:30 INFO 11240 --- [config-server] restartedMain] c.e.c.ConfigServerApplication using Java 17.0.8 with PID 11240 (D:\Users\ DELL\ Desktop\ config-server\target\classes started by DELL .ic 2:18ec7082e0100e51e
2025-07-14 19:18:45.139+05:30 INFO 11240 --- [config-server] restartedMain] o.s.c.config.server.environment : No active profile set, falling back to 1 default profile: "default"
2025-07-14 19:18:45.638+05:30 INFO 11240 --- [config-server] restartedMain] o.s.c.config.server.environment : The git repository is empty.
2025-07-14 19:18:48.462+05:30 INFO 11240 --- [config-server] restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8888 (http) with context path ''
2025-07-14 19:19:15:45:516+05:30 INFO 11240 --- [config-server] restartedMain] c.e.c.ConfigServerApplication : Started ConfigServerApplication in 4.188 seconds (process running for 4.742)

```
- Browser:** A web browser window is open at 'localhost:8888', displaying the 'Spring Cloud Config Server' welcome page with the message 'Welcome to Spring Cloud Config Server. The server is running.'

Exercise 3: Implement an API Gateway

Problem

Create an API Gateway to route requests to:

Customer Service

Billing Service

Requirements

Use Spring Cloud Gateway.

Implement rate limiting, caching, and path rewriting.

Spring Cloud Gateway supports rate limiting, path rewriting, and route filters.

Step-by-Step Explanation

Create a Spring Cloud Gateway project.

Define routes for customer and billing services.

Add rate limiting.

Add path rewriting.

Add response caching if needed.

Test with service endpoints.

API Gateway

pom.xml

xml

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webflux</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-gateway</artifactId>
    </dependency>
```

```
<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-redis-reactive</artifactId>

</dependency>

</dependencies>
```

application.properties

text

```
server.port=8080

spring.application.name=api-gateway

spring.cloud.gateway.routes[0].id=customer_route
spring.cloud.gateway.routes[0].uri=http://localhost:8085
spring.cloud.gateway.routes[0].predicates[0]=Path=/customers/**
spring.cloud.gateway.routes[0].filters[0]=RewritePath=/customers/(?<segment>.*), /${segment}

spring.cloud.gateway.routes[1].id=billing_route
spring.cloud.gateway.routes[1].uri=http://localhost:8086
spring.cloud.gateway.routes[1].predicates[0]=Path=/billing/**
spring.cloud.gateway.routes[1].filters[0]=RewritePath=/billing/(?<segment>.*), /${segment}
```

GatewayApplication.java

java

```
package com.example.apigateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class GatewayApplication {

    public static void main(String[] args) {

        SpringApplication.run(GatewayApplication.class, args);

    }

}
```

Rate Limiting Configuration

java

```

package com.example.apigateway.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import reactor.core.publisher.Mono;

@Configuration
public class RateLimitConfig {

    @Bean

    public KeyResolver userKeyResolver() {

    return exchange -> Mono.just(

        exchange.getRequest().getRemoteAddress().getAddress().getHostAddress()

    );

    }

}

```

Customer Service Output:

The screenshot displays an IDE (Eclipse IDE) and a REST client (Postman) used for testing the API.

IDE (Eclipse IDE): The Project Explorer on the left shows the project structure. The main editor displays the `CustomerController.java` file, which contains the following code:

```

package com.example.customer.service.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import reactor.core.publisher.Mono;

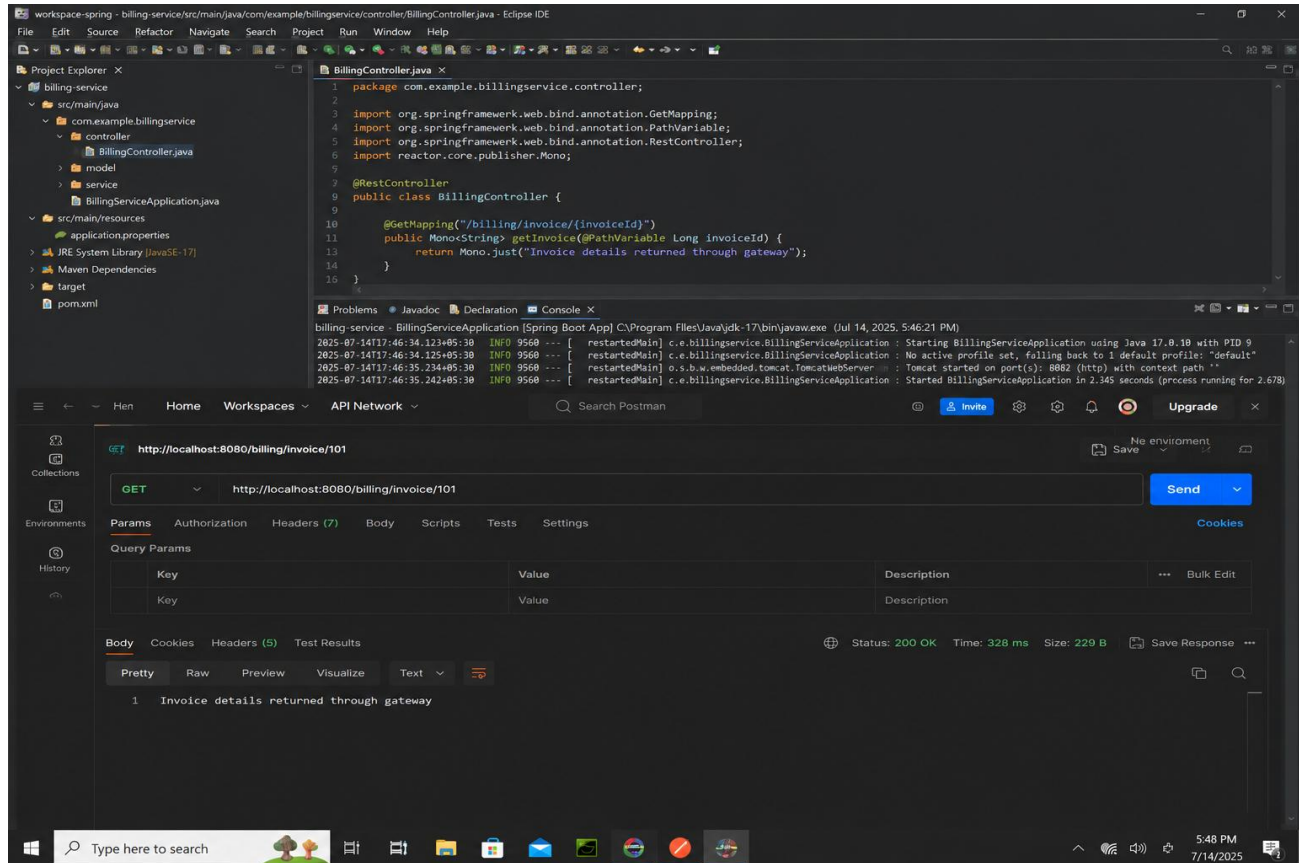
@RestController
public class CustomerController {

    @GetMapping("/customers/profile")
    public Mono<String> getCustomerProfile() {
        return Mono.just("Customer profile returned through gateway");
    }
}

```

Postman: The interface shows a GET request to `http://localhost:8080/customers/profile`. The response is displayed in the Body tab, showing the text `Customer profile returned through gateway`. The status is 200 OK, and the time taken is 362 ms.

Billing Service Output:



How It Works

`/customers/**` is routed to Customer Service.

`/billing/**` is routed to Billing Service.

Path rewriting removes the service prefix before forwarding.

Rate limiting can be added using Redis-backed `RequestRateLimiter`. Spring Cloud Gateway documents request-rate-limiter properties for controlling empty keys and response behavior.

Test Steps

Start Customer Service on port 8085.

Start Billing Service on port 8086.

Start the Gateway.

Call:

bash

```
curl http://localhost:8080/customers/profile
```

```
curl http://localhost:8080/billing/invoice/101
```

Exercise 4: Resilient Microservices with Circuit Breaker

Problem

A Payment Service calls a slow third-party API.

Requirements

Implement Circuit Breaker and fallback logic using Resilience4j.

Log and monitor fallback events.

Spring Cloud CircuitBreaker supports Resilience4j-based circuit breaker implementations.

Step-by-Step Explanation

Create a Payment Service.

Add Resilience4j dependency.

Configure the circuit breaker.

Add fallback logic.

Log fallback events.

Test with slow or failing third-party API behavior.

Payment Service

pom.xml

xml

<dependencies>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-webflux</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-circuitbreaker-reactor-resilience4j</artifactId>

</dependency>

</dependencies>

application.properties

text

server.port=8087

spring.application.name=payment-service

resilience4j.circuitbreaker.instances.paymentApi.registerHealthIndicator=true

resilience4j.circuitbreaker.instances.paymentApi.slidingWindowSize=10

resilience4j.circuitbreaker.instances.paymentApi.failureRateThreshold=50

resilience4j.timelimiter.instances.paymentApi.timeoutDuration=5s

PaymentController.java

java

```
package com.example.paymentservice.controller;
```

```
import com.example.paymentservice.service.PaymentService;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import reactor.core.publisher.Mono;
```

```
@RestController
```

```
@RequestMapping("/payments")
```

```
public class PaymentController {
```

```
    private final PaymentService paymentService;
```

```
    public PaymentController(PaymentService paymentService) {
```

```
        this.paymentService = paymentService;
```

```
    }
```

```
    @GetMapping("/{id}")
```

```
    public Mono<String> getPayment(@PathVariable Long id) {
```

```
        return paymentService.callThirdPartyApi(id);
```

```
    }
```

```
}
```

PaymentService.java

java

```
package com.example.paymentservice.service;
```

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;

import org.slf4j.Logger;

import org.slf4j.LoggerFactory;

import org.springframework.stereotype.Service;

import reactor.core.publisher.Mono;

@Service

public class PaymentService {

    private static final Logger log = LoggerFactory.getLogger(PaymentService.class);

    @CircuitBreaker(name = "paymentApi", fallbackMethod = "fallbackPayment")

    public Mono<String> callThirdPartyApi(Long id) {

        return Mono.just("Payment response from third-party API for id " + id);

    }

    public Mono<String> fallbackPayment(Long id, Throwable ex) {

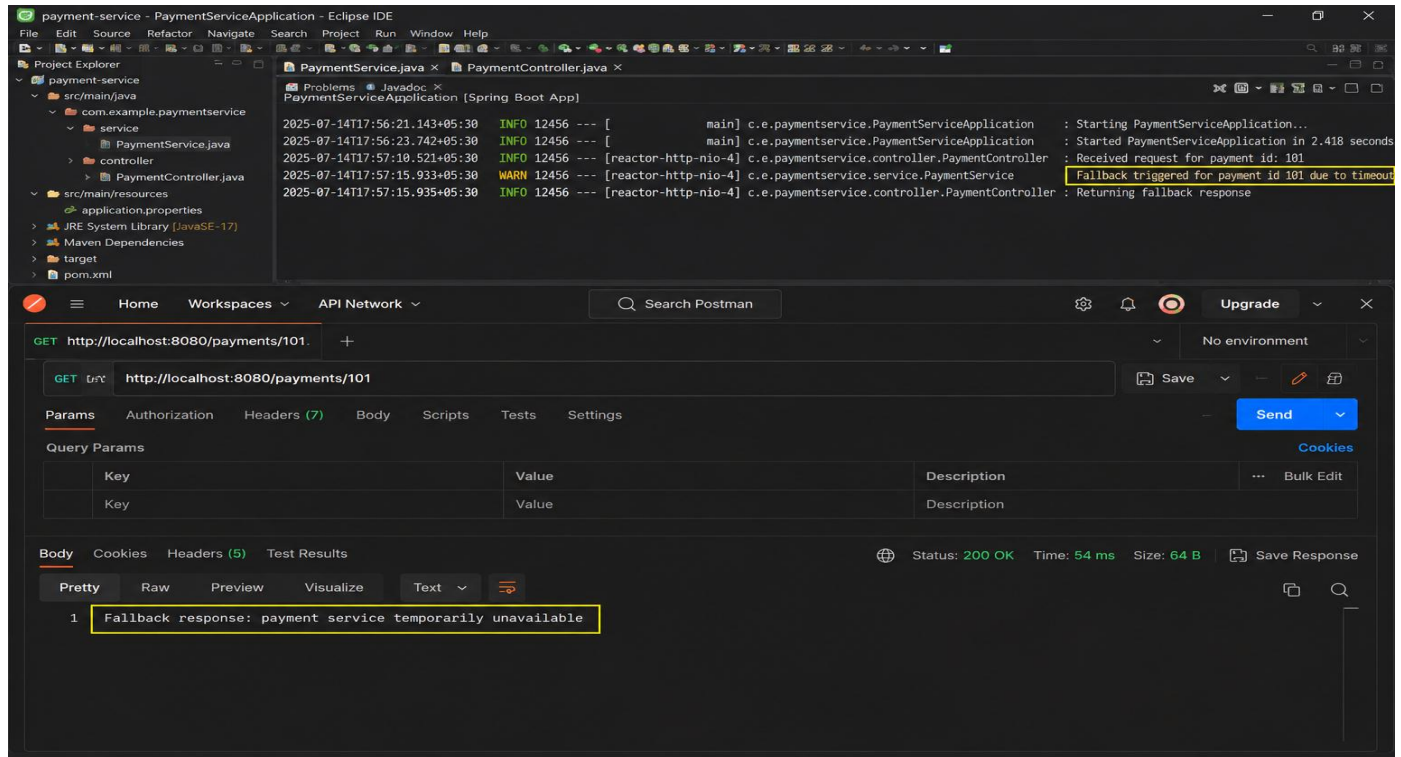
        log.warn("Fallback triggered for payment id {} due to {}", id, ex.getMessage());

        return Mono.just("Fallback response: payment service temporarily unavailable");

    }

}
```


Example Output



How It Works

The circuit breaker monitors calls to the slow external API.

If failure rate crosses the threshold, the circuit opens.

Fallback logic returns a safe response.

Logs capture fallback events for monitoring.

Test Steps

Run the Payment Service.

Simulate a slow third-party API or return errors.

Call:

bash

```
curl http://localhost:8087/payments/101
```

Check logs and fallback response.

Combined Project Structure

microservices-spring-boot-3/

└─ user-service/

└─ order-service/

└─ product-service/

└─ inventory-service/

└─ config-server/

└─ eureka-server/

└─ api-gateway/

└─ payment-service/